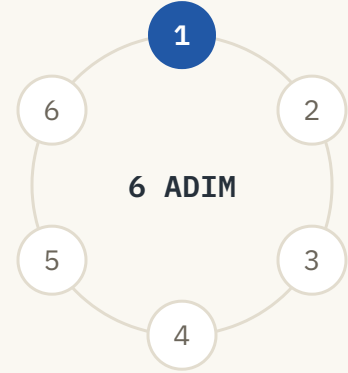


Sensörsüz 3 Fazlı BLDC Motor Komütasyon Simülasyonu

Half-bridge kavramından 6 adımlı komütasyona; state machine simülasyonundan UART telemetrisine ve PC izleme arayüzüne.



Üç Aşamalı Yol Haritası

FAZ 1

State machine + half-bridge takibi

6 adımı yazılımda doğru
üret, GPIO ile ölçülebilir kıl



FAZ 2

UART ile durum telemetrisi

Her adım değişimini PC'ye
metin olarak raporla



FAZ 3

PC tarafında izleme arayüzü

Terminal → Python → canlı
görsel gösterim



İLERİ SEVİYE

Gerçek motor + sensörsüz BEMF

Donanım geldiğinde;
haritada yerini bil

Neden önce donanımsız?

Gerçek bir 3 fazlı inverter kartında yanlış sinyal MOSFET veya sürücü IC yakar. Mantığı önce yazılımda kurup GPIO pinlerinden multimetre ile doğrulamak, en ucuz hata ayıklama aşamasıdır.

BLDC Motorun Çalışma Mantığı

Fırçalı DC motor

Komütasyonu fırçalar mekanik temasla, kendiliğinden yapar. Kolay ama fırçalar aşınır, kıvılcım ve gürültü üretir.

BLDC (fırçasız)

Rotor kalıcı mıknatıs, stator 3 sargı (A, B, C). Fırça yok — hangi sargının ne zaman enerjileneceğine yazılım karar verir. Komütasyon artık bizim sorumluluğumuzda.

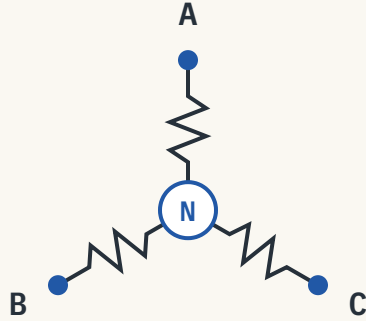
Rotor = dönen mıknatıs · Stator = sabit A, B, C sargıları

SOMUT DÜŞÜN: SALINCAK

Rotor mıknatısı salıncaktaki çocuk, sargılar iten eller.

Doğru anda, doğru elle itersen salıncak düzenli döner; erken ya da geç itersen yavaşlar, hatta durur. 6 adımlı komütasyon, bu "doğru anda itme" sırasının ta kendisidir.

Y Baęlantılı Stator ve Temel Kavramlar



Üç sargının bir ucu ortak nötr (N) noktasında buluşur; A, B, C uçları birer half-bridge çıkışına gider.

Sargı (faz) A/B/C

Statordeki 3 elektrik hattı; her biri bir half-bridge'e bağlanır

Nötr noktası (N)

3 sargının ortak iç noktası; gerçek veya sanal olarak ölçülebilir

Elektriksel derece

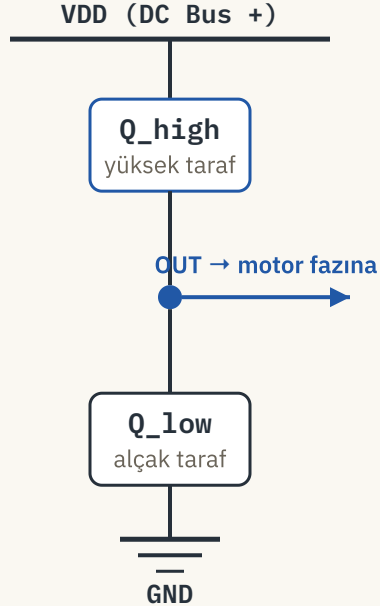
Bir komütasyon döngüsü = 360° elektriksel; kutup sayısına göre mekanik dönüşten farklı olabilir

Trapezoidal komütasyon

Her an yalnız 2 faz sürülür (biri yüksek, biri düşük), 3. faz boşta — "6 adım" yöntemi

FOC (vektör kontrolü) çok daha pürüzsüzdür ama Clarke/Park dönüşümleri ister — önce bu temeller oturmalı.

Half-Bridge: En Küçük Yapı Taşı



YÜKSEK

Q_high ON, Q_low OFF → OUT = VDD (kaynak akım)

DÜŞÜK

Q_low ON, Q_high OFF → OUT = GND (akım yutar)

FLOAT

İkisi de OFF → OUT yüksek empedans (Hi-Z) — BEMF burada ölçülür

Somut düşün: Üç konumlu bir anahtar — yukarı = artı uca, aşağı = eksiye, orta = hiçbir yere bağlı değil (boşta).

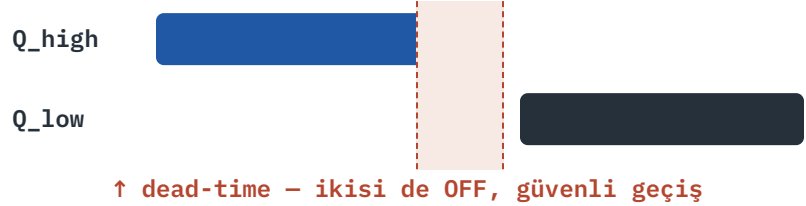
Hız/tork kontrolü Q_high'ı PWM ile anahtarlayarak yapılır: duty ne kadar yüksekse faza uygulanan ortalama gerilim o kadar büyük — PC13 LED parlaklık ayarıyla aynı prensip.

Shoot-Through Tehlikesi ve Dead-Time

Shoot-through nedir?

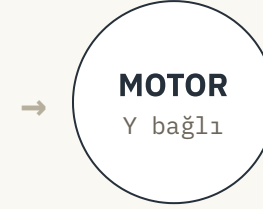
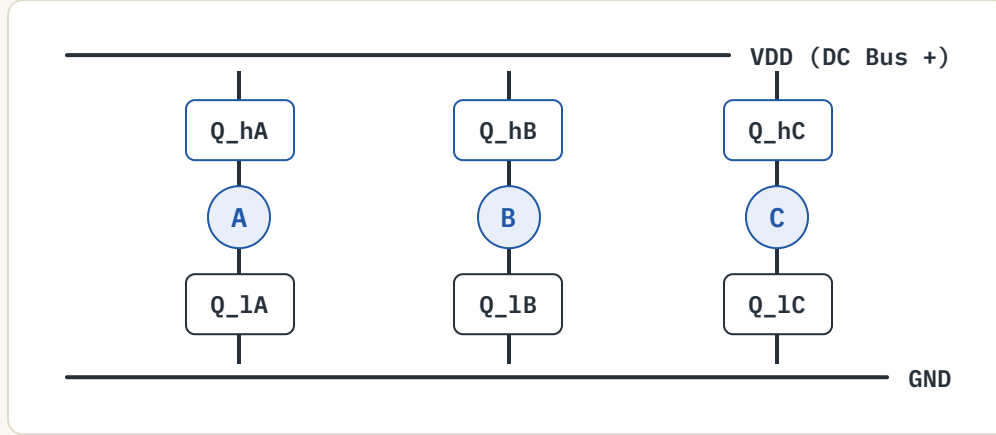
Q_high ve Q_low aynı anda ON olursa, VDD'den GND'ye — motor sargısına hiç uğramayan — doğrudan bir kısa devre yolu açılır. Akımı sınırlayan hiçbir şey yoktur; MOSFET'ler saniyeler içinde aşırı akımla yanar.

Somut düşün: pilin + ve – ucunu kalın bir telle birleştirmek gibi — enerji işe değil, ısıya gider.



STM32'de: TIM1 (advanced-control) complementary PWM + donanımsal dead-time (BDTR / DTG) üretir. TIM2 bunu yapamaz — gerçek motor sürüşünde TIM1 şart.

Üç Fazlı İnverter: 3 Half-Bridge, 6 MOSFET



Her sütun, bir önceki slayttaki half-bridge'in aynısı. Üçü de aynı DC bus'ı (VDD / GND) paylaşır: $3 \times 2 = 6$ MOSFET.

Gerçek donanıma geçince: 6 MOSFET veya entegre sürücü (örn. DRV8305/DRV8353), gate sürme, koruma, TIM1 dead-time. **Şimdilik hiçbirisi gerekmiyor — FAZ 1 tamamen yazılımda.**

6 Adımlı Komütasyon Tablosu

ADIM	FAZ A	FAZ B	FAZ C
STEP 1	YÜKSEK	DÜŞÜK	FLOAT
STEP 2	YÜKSEK	FLOAT	DÜŞÜK
STEP 3	FLOAT	YÜKSEK	DÜŞÜK
STEP 4	DÜŞÜK	YÜKSEK	FLOAT
STEP 5	DÜŞÜK	FLOAT	YÜKSEK
STEP 6	FLOAT	DÜŞÜK	YÜKSEK

Her adımda **tam 1 faz FLOAT** kalır — float faz sırayla $C \rightarrow B \rightarrow A \rightarrow C \rightarrow B \rightarrow A$ döner.

Ardışık adımlar arasında **yalnız 2 fazın durumu değişir**; üçüncüsü sabit kalır.

STEP 6'dan sonra STEP 1'e dönülür: **6 adım = 360° elektriksel** = bir tam döngü.

Yön (CW/CCW) sargı bağlantısına göre değişebilir; buradaki atamalar örnek kabuldür — önemli olan desen.

Komütasyon Döngüsü: 360° Elektriksel



A

FLOAT

B

YÜKSEK

C

DÜŞÜK

Tabloyu ezberleme, deseni gör: aktif çift saat yönünde kayar, boşta kalan faz her adımda bir sonrakine devreder.

GPIO ile Üç Durumun Taklidi

YÜKSEK

`GPIO_MODE_OUTPUT_PP`
`WritePin(..., SET)`

Pin $\approx 3.3V$, kaynak akım.
Q_high ON durumunu temsil eder.

DÜŞÜK

`GPIO_MODE_OUTPUT_PP`
`WritePin(..., RESET)`

Pin $\approx 0V$, akım yutar.
Q_low ON durumunu temsil eder.

FLOAT

`GPIO_MODE_INPUT`
`Pu11 = GPIO_NOPULL`

Pin yüksek empedans (Hi-Z).
İki MOSFET'in de OFF halini temsil eder.

Somut düşün: FLOAT, fişi çekilmiş kablo gibidir — ne artıya ne eksiye bağlı, sadece boşta.

Böylece "kâğıt üzerinde doğru" değil, **multimetreyle pinden ölçülebilir** bir sinyal üretirsin — ileride aynı 3 sinyal gerçek gate driver IC'sine gidecek.

Enum ve Arama Tablosu

```
typedef enum {  
    PHASE_FLOAT = 0,  
    PHASE_LOW,  
    PHASE_HIGH  
} PhaseState_t;  
  
typedef enum {  
    MOTOR_STEP_1 = 0,  
    /* ... MOTOR_STEP_6'ya kadar */  
} MotorStep_t;  
  
const PhaseState_t commutation_table[6][3] = {  
    { PHASE_HIGH,  PHASE_LOW,  PHASE_FLOAT }, // STEP 1  
    { PHASE_HIGH,  PHASE_FLOAT, PHASE_LOW  }, // STEP 2  
    { PHASE_FLOAT, PHASE_HIGH,  PHASE_LOW  }, // STEP 3  
    { PHASE_LOW,   PHASE_HIGH,  PHASE_FLOAT }, // STEP 4  
    { PHASE_LOW,   PHASE_FLOAT, PHASE_HIGH  }, // STEP 5  
    { PHASE_FLOAT, PHASE_LOW,   PHASE_HIGH  }, // STEP 6  
};
```

Nasıl okunur?

Satır = adım, sütun = faz (0: A, 1: B, 2: C). Bu tablo, bir önceki slayttaki tablonun birebir C karşılığıdır.

Tanıdık desen

LED projesindeki LedState_t state machine'inin aynısı — sadece 4 yerine 6 durum ve her durumda 3 fazlık hedef var.

Henüz yazma

Kod tamamen kavramsal. main.c'ye eklemeyen önce GPIO pin seçimi, CubeMX ayarları ve geçiş timer'ı birlikte planlanmalı.

Zamanlama ve Donanımsız Doğrulama

TIM kesmesi — örn. her 10 ms'de bir tetiklenir ("elektriksel hız")



```
current_step = (current_step + 1) % 6;
```



commutation_table[current_step] satırını oku → 3 GPIO'yu YÜKSEK / DÜŞÜK / FLOAT yap

Adım süresi kısaldıkça "motor" hızlanır — donanım yokken bu yalnızca simülasyon hızıdır.

DOĞRULAMA YÖNTEMLERİ

Debugger watch

current_step ve tablo satırı adım adım izlenir

PC13 blink-code

LED her adımda adım numarası kadar kısa yanıp söner

Multimetre / osiloskop

Pinde gerçekten 3.3V / 0V / Hi-Z görüldüğü doğrulanır

UART

En rahat ve kalıcı yöntem — sıradaki bölüm: FAZ 2

UART ile Durum Telemetrisi

CUBEMX AYARLARI

Periferik	USART2
TX pini	PA2
RX pini (yalnız gönderimde şart değil)	PA3
Baud rate	115200
Çerçeve	8N1

Black Pill'de dahili USB-seri köprüsü yok — harici USB-TTL adaptör gerekir (CP2102, CH340, FT232). Bonus: `_write()` fonksiyonunu `HAL_UART_Transmit`'e yönlendirirsen `printf` doğrudan UART'a yazar.

SERİ PORT ÇIKTISI — HER ADIM DEĞİŞİMİNDE BİR SATIR

STEP:1 A:HIGH B:LOW C:FLOAT

STEP:2 A:HIGH B:FLOAT C:LOW

STEP:3 A:FLOAT B:HIGH C:LOW

...

Protokol karmaşık olmak zorunda değil: insan okuyabilsin, yeter. Faz 3'te ayrıştırmayı kolaylaştırmak istersen CSV alternatifi: **1,H,L,F**

Kesme Kuralı: Bayrak Deseni

TIM ISR – KISA TUT

Adımı ilerlet + bayrağı set et

UART gönderimi YOK

`main while(1)`

Bayrağı kontrol eder

Set ise gönderime geçer

`HAL_UART_Transmit`

Durum satırını gönderir

Bloklasa da kesmeleri engellemez

Neden? HAL_UART_Transmit varsayılan olarak blocking'dir — kesme içinde çağrılırsa ISR'ı milisaniyelerce kilitler, diğer kesmeler gecikir.

Somut düşün: kapı zili çaldığında kapıda iş yapılmaz — sadece "geldi" notu düşülür, paket sonra alınır.

İleri seviye: HAL_UART_Transmit_IT veya _DMA — CPU hiç bloklanmaz. İlk sürüm için bayrak deseni yeterli.

PC Tarafında İzleme Arayüzü

1

Terminal programı — PuTTY, Tera Term, VS Code Serial Monitor

Ham metni gör, protokolün doğru çalıştığını onayla

ÇOK KOLAY

2

Python + pyserial — konsola yazdırma

Satırları oku, ayrıştır (parse), ayrıştırılmış hâlde konsola yaz

KOLAY

3

Python GUI — tkinter / PyQt

6 dairesel canlı gösterim: bu sunumdaki döngü diyagramının gerçek zamanlı hâli

ORTA

4

Web tabanlı — yerel sunucu + WebSocket + HTML, ya da STM32CubeMonitor

Tarayıcıda görselleştirme; hazır alternatif olarak CubeMonitor

ORTA-İLERİ

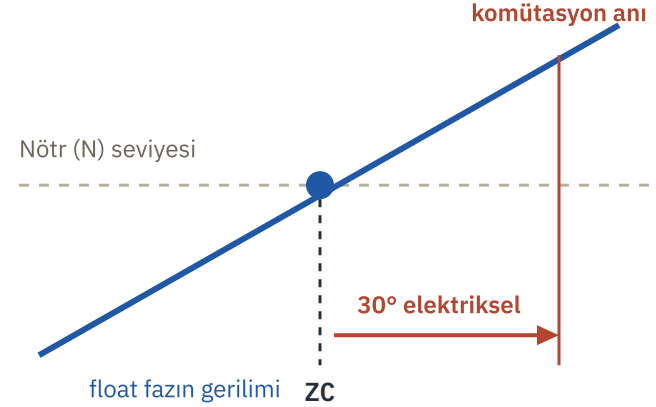
Önerilen sıra yukarıdan aşağı: her katman bir öncekinin doğru çalıştığını kanıtlar.

BEMF Sıfır Geçişi

Float bırakılan faz pasif değildir: dönen rotor mıknatısları o sargıda bir gerilim indükler — **Back-EMF**. Bu gerilim, adımın ortasına yakın nötr (N) seviyesini keser: **sıfır geçişi (ZC)**.

ZC'yi yakaladığın an, bir sonraki komütasyonun tam **30° elektriksel** sonra gelmesi gerektiğini bilirsin — motor kendi konumunu kendisi söyler.

Somut düşün: dönen motor aynı zamanda bir dinamodur. Boştaki sargı, rotorun nerede olduğunu fısıldayan bedava bir sensöre dönüşür — ek donanım yok, sinyal motorun kendisinden.



Sensörsüz Sürüşün Gereksinimleri

TIM1

Advanced-control timer: complementary PWM kanalları + donanımsal dead-time (BDTR/DTG) — TIM2'nin yapamadığı iş.

Senkronize ADC

Float fazın gerilimini anahtarlama gürültüsünden uzak, TIM1 tetiklemesiyle tam doğru anda örneklemek.

Blanking (maskeleme)

Her komütasyondan hemen sonra ZC tespitini kısa süre yok saymak — anahtarlama çınlaması (ringing) yanlış tetiklemesin.

Startup ramp

Duran motorda BEMF = 0. Önce açık çevrim, artan hızlı komütasyonla döndür; yeterli BEMF oluşunca sensörsüz moda geç.

Bu aşama şimdi değil: Faz 1-2-3'ü tamamlamadan buraya atlama — 6 adım ve half-bridge mantığını sağlam oturtmak, BEMF algılamayı anlayanın ön koşulu.

ÖZET

Hızlı Tekrar

- ✓ Half-bridge = 2 MOSFET; 3 çıkış durumu: YÜKSEK / DÜŞÜK / FLOAT.
- ✓ 3 fazlı inverter = 3 half-bridge = 6 MOSFET, ortak DC bus.
- ✓ GPIO OUTPUT_PP ↔ INPUT (NOPULL) geçişiyle 3 durum donanımsız taklit edilir.
- ✓ FAZ 2 = USART2 (PA2/PA3), 115200-8N1; ISR bayrak set eder, gönderim main'de yapılır.
- ✓ Shoot-through riski → dead-time şart; TIM1 (BDTR/DTG) donanımda üretir, TIM2 üretmez.
- ✓ 6 adım: her adımda 1 faz YÜKSEK, 1 DÜŞÜK, 1 FLOAT; float faz C→B→A sırasıyla döner.
- ✓ FAZ 1 = enum + commutation_table[6][3] + timer; doğrulama: watch, blink-code, multimetre.
- ✓ FAZ 3 = terminal → pyserial → GUI; ileri seviye = BEMF ZC + 30° gecikme.

Quiz: Kavram Soruları I

SORU 1

Q_high ve Q_low aynı anda ON olursa ne olur?

- A Motor daha hızlı döner
- B VDD'den GND'ye kısa devre oluşur; MOSFET'ler yanabilir
- C Faz çıkışı FLOAT durumuna geçer

SORU 2

6 adımlı komütasyonda her adımda kaç faz aktif sürülür?

- A 2 — biri YÜKSEK, biri DÜŞÜK; üçüncüsü boşta
- B 3 — üç faz da her an enerjili
- C 1 — her seferinde tek sargı

Quiz: Kavram Soruları II

SORU 3

FLOAT durumunu GPIO ile nasıl taklit edersin?

- A** GPIO_MODE_INPUT + Pull = NOPULL
- B** OUTPUT_PP + WritePin(..., RESET)
- C** INPUT + Pull-up direnci aktif

SORU 4

Gerçek motor sürüşünde neden TIM2 değil TIM1 gerekir?

- A** TIM1'in kesme önceliği daha yüksektir
- B** TIM1 daha hızlı sayabilir
- C** Complementary PWM + donanımsal dead-time yalnız TIM1'de var

Quiz: Kod Okuma I

KOD 1

```
current_step = (current_step + 1) % 6;
```

- A Adımı 1 ilerletir; 5'ten sonra 0'a sarar (STEP 6 → STEP 1)
- B current_step'i 6 ile çarpar, kalanı atar
- C Adımı rastgele bir değere ayarlar

KOD 2

```
PhaseState_t s =  
    commutation_table[current_step][2];
```

- A Faz B'nin hedef durumunu okur
- B Bu adımda Faz C'nin hedef durumunu okur
- C STEP 2 satırının tamamını okur

Quiz: Kod Okuma II

KOD 3 – BİR TIMER ISR'ININ İÇİNDE

```
HAL_UART_Transmit(&huart2, msg,  
                  len, HAL_MAX_DELAY);
```

Buradaki sorun ne?

- A UART kesme içinde donanımsal olarak çalışamaz
- B Blocking'dir: gönderim bitene dek ISR kilitli kalır, diğer kesmeler gecikir
- C Baud rate otomatik olarak düşer

KOD 4 – PİN OUTPUT_PP MODUNDA

```
HAL_GPIO_WritePin(GPIOA, FAZ_A_Pin,  
                  GPIO_PIN_SET);
```

Bu çağrı ne yapar?

- A Pini $\approx 3.3V$ 'a çeker — half-bridge'de Q_high ON (YÜKSEK) taklidi
- B Pini Hi-Z (FLOAT) durumuna alır
- C Pini GND'ye çeker (DÜŞÜK)

KAPANIŞ

Sonraki Adım

1 CubeMX planı: hangi 3 GPIO pini Faz A / B / C olacak

2 Adım timer'ının seçimi ve yapılandırması — LED fade projesindeki TIM2 deseni

3 main.c USER CODE bölümlerine birlikte ekleme — LED/buton projesindeki akışın aynısı

Önce mantık → sonra telemetri → sonra görselleştirme. Güç elektroniği en sona.